Private Dataset Workspace

Moneer | Software Security Assignment I | 22 October 2026

1 Purpose and scope

The Private Dataset Workspace is a small Python application that demonstrates how security can fail when separately running services are combined. Users can create private numeric datasets, inspect their own records and request a CSV export. The final implementation rejects the four planted attacks investigated in this report. [1]

Ali and John are ordinary seeded users. Admin has the additional ability to inspect global dataset metadata, including owner and title. This privilege does not grant access to another user's numeric rows or CSV results.

The functional requirements are login, dataset creation and search, private record retrieval, CSV export and retrieval, and administrative metadata inspection. Export involves all three services. The principal security requirements are to authenticate every protected operation, enforce ownership at the resource boundary, bind delegated authority to a specific operation, reject malformed input, and reject unapproved dependency artifacts. The requirements, assets, access matrix and STRIDE analysis are in docs/security_design.md.

A dataset contains a title of at most 80 characters and between one and 100 integers. Each integer must be between minus one million and one million. These deliberately narrow constraints bound legitimate work and remove arbitrary text cells from the CSV format. The application has no registration, email, payments, public hosting or availability target. The optional malicious-helper and supply-chain experiments were not attempted. A separate installed helper package is nevertheless included because that architecture requirement is mandatory.

The development sequence is preserved in Git rather than reconstructed as an informal narrative. Commit faf9582 records the lifecycle plan before feature code. Commit dcabff9 introduces the services and four deliberate weaknesses; tag vulnerable-lab identifies that milestone. Commit 716e188 fixes the security boundaries.

Role	Permitted data access
Ali and John	Own datasets and own CSV results
Admin	Own data plus global owner and title metadata
Processor	One delegated export over its HTTP interface

2 Components and trust boundaries

The Application API, Resource Service and Processing Service run as three independent uvicorn processes and communicate through HTTP JSON interfaces. The launcher assigns separate ports and process environments. SQLite is storage owned by Resource, not a fourth application component. The test fixture starts new processes with a temporary database and verifies distinct process identifiers.

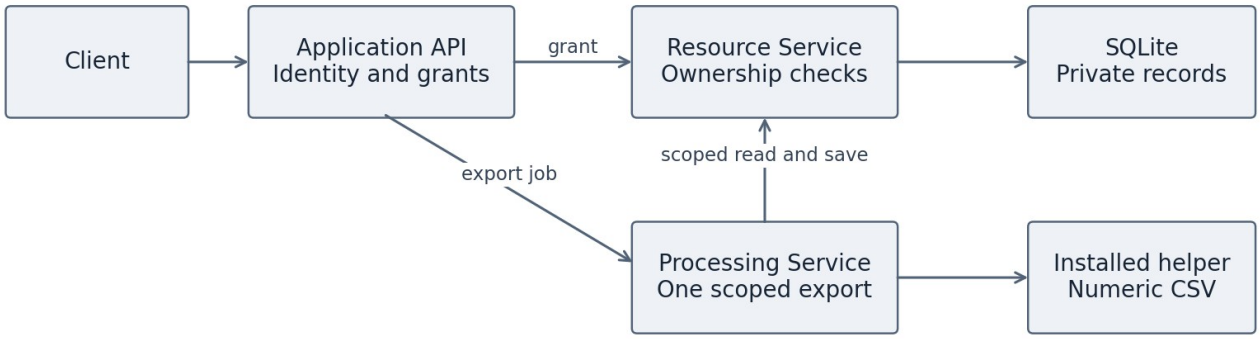


Figure 1 Component boundaries and the installed helper

API receives the password, verifies a salted scrypt digest and issues a user session. It coordinates requests but never opens the dataset database. Resource owns the SQLite datasets and results tables and makes the final ownership decisions. Processor receives an export job, retrieves authorized numeric rows, calls the installed workspace-csv helper, and sends the generated CSV back to Resource. API can subsequently retrieve the result for its owner.

The helper is a separate Python package with its own pyproject.toml, source directory and distributable wheel. requirements.lock declares workspace-csv version 1.0.0 and installs it from its local package folder alongside the approved third-party dependencies. Processor imports that installed package; its implementation is not pasted into the service.

The key trust boundaries are client to API, API to each downstream service, Processor to Resource, and the dependency installation boundary. Fixed configured URLs prevent callers from selecting arbitrary network destinations. Requests have five-second timeouts, do not follow redirects and do not inherit proxy settings. Resource validates returned CSV against its own expected serialization, preventing an accepted processing job from silently storing altered numeric output.

Separate processes are not a complete sandbox. They share one operating-system account in this local prototype, so a fully compromised process could attempt filesystem access outside its intended role. Environment minimization removes Resource keys and database paths from Processor's configuration, but does not establish OS access control. Production needs separate identities, filesystem permissions and network controls before stronger isolation claims are justified.

3 Identity and delegated authority

Authentication uses established mechanisms rather than custom cryptography. Password verification uses Python's scrypt implementation with a random salt for every seeded account. A successful login returns a signed JWT created through PyJWT. The session names the user, has audience api, carries action session and expires after 15 minutes. API checks the token and resolves the role from its own account store, so a caller cannot choose an administrative role in request data [7].

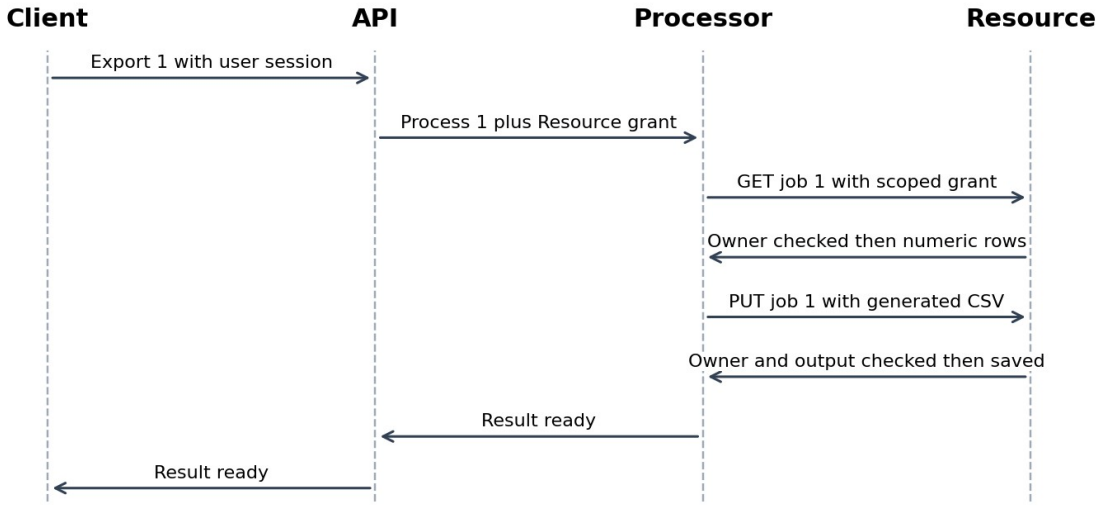


Figure 2 Authority through the three service export workflow

API creates a fresh Resource grant for each downstream operation. The signed claims carry subject, issuer, receiving service, action, dataset identifier, issue time, expiry and a token identifier. Collection operations use resource zero. Resource requires these claims, validates the signature with a fixed algorithm allowlist and checks the requested action and identifier before consulting ownership. Grant validity is one minute.

Export adds a Processor grant for action process and the same dataset identifier. Processor relays the Resource export grant; it cannot issue that grant because it lacks the Resource signing key. Resource authorizes both GET jobs and PUT jobs independently. The result is stored under the authenticated subject, and result retrieval filters by that subject. This prevents a service from treating its participation in a workflow as permission to access every user's records.

Three random signing keys separate user sessions, Resource grants and Processor grants. API and Resource share one trusted signing domain; API and Processor share another. A compromised Resource or API is therefore beyond the selected containment boundary. Grants are reusable until expiry, not one-use capabilities. Restart rotates the in-memory keys and invalidates sessions. Production would need persistent key management, revocation and preferably asymmetric service signatures to separate signing authority from verification.

4 Vulnerabilities and investigation

The catalog provides three planted weaknesses: SQL injection, missing authorization and improper input validation [2]. A fourth weakness arises specifically in the export composition. The same four security assertions were executed against the vulnerable milestone and the fixed implementation. The original run failed all four assertions, with actual responses saved in evidence/vulnerable-tests.txt and its JUnit file.

V1 is CWE-89 in Resource's search endpoint. The original implementation concatenated the authenticated subject and search string into SQL. Ali submitted the search string shown in the evidence table and received metadata belonging to both Ali and John. The fix binds the owner and search pattern as SQL parameters. The regression now checks that every returned row belongs to Ali, including when the search contains SQL metacharacters.

V2 is CWE-862 on ordinary dataset reads. The original route accepted a valid grant for the requested identifier and fetched the record without comparing its owner with the subject. Ali's request for dataset 2 returned John's values with HTTP 200. The fixed route calls owned before returning rows. It returns the same unavailable response for absent and foreign datasets, reducing direct existence disclosure. Admin is subject to the same row-ownership rule.

V3 is CWE-20. The original dataset schema declared a list of integers without enforcing the documented 100-row quota. A request containing 101 rows was accepted with HTTP 201 and persisted. This demonstrates a security-relevant quota bypass and a precondition for excess storage or processing work; it does not demonstrate an actual service outage. The fix applies a bounded strict schema at both API and Resource, including integer ranges and unknown-field rejection. A direct Resource test confirms that bypassing API does not bypass validation.

V4 is a confused-deputy failure in service composition. API legitimately starts an export, but the original Resource jobs route assumed that a service-mediated request was already authorized for its subject. Ali could export dataset 2 and receive a ready result generated from John's rows. Fixing V2 alone would leave this separate jobs route exploitable. The correction checks ownership at job read and again at job save, while keeping subject, operation, dataset and expiry bound to the delegated grant. Result retrieval and output validation add further checks.

The observed cross-user disclosures receive high qualitative priority because an ordinary authenticated user can trigger them directly. The quota bypass receives medium priority in this local synthetic setting. These are contextual judgments, not computed CVSS scores. This follows the course emphasis on attack conditions and violated assumptions and the supplied prioritization reading's emphasis on context [4, 6]. The final branch has no runtime switch that re-enables the flaws; the vulnerable tag preserves them for controlled review.

Probe	Vulnerable observation	Fixed observation
V1 SQL search	' OR 1=1 -- exposes both owners	No foreign metadata
V2 Foreign read	200 with John values	404 unavailable
V3 Row quota	101 rows accepted with 201	422 rejected
V4 Foreign export	200 ready for dataset 2	404 unavailable

5 Design principles and lifecycle decisions

Least privilege appears in both the roles and service grants. Admin can inspect metadata but cannot read foreign rows. Processor has no Resource signing key or account store and receives authority for only the current dataset export. Its key cannot authenticate a Resource request. Tests exercise that attempted forgery and mismatched resource scopes. [3]

Complete mediation means that the resource owner checks every relevant entry point. Resource authorizes ordinary reads, delegated job reads, job saves and result retrieval separately. It does not rely solely on API checks or trust a request because it originates from another service. Independent tests call internal endpoints with correctly signed but inappropriate grants. This is the principal lesson from V4: a secure ordinary read route does not automatically make the composed export secure.

Fail-safe defaults appear in missing-credential rejection, expiry and audience validation, strict input schemas and downstream error handling. A timeout or rejected downstream request stops the operation instead of returning foreign data or silently retrying with broader privileges. The tests cover missing credentials, invalid signatures, absent claims, expired tokens, wrong actions and wrong audiences. Keeping the domain small is a supporting design choice: fixed numeric CSV output avoids arbitrary filenames, browser rendering and general-purpose command execution.

The initial plan covers requirements, implementation and build, testing, and release/deployment/operation. Its requirements gate identifies assets and authorities before implementation. The later STRIDE matrix connects spoofing to token checks, tampering to parameterized SQL and validation, disclosure to ownership, elevation to roles, and availability to bounded inputs and timeouts. Metadata denial logs support investigation, although durable audit completeness and tamper resistance remain operational work. This is a concrete application of the course's asset-to-threat-to-control-to-evidence chain [5].

The plan also determines gate decisions. The vulnerable milestone is retained as investigation evidence but fails release requirements. Confirmed authorization and validation violations must be fixed before the final candidate passes. Packaging was later simplified: third-party packages are downloaded through a hash lock, while the required helper remains a local installable artifact. The release checks still reject unapproved bytes and installed dependencies.

Python's managed runtime reduces common manual-memory errors discussed in the course, but does not determine which user owns a record. FastAPI and Pydantic enforce the declared request schema; PyJWT handles signatures and registered claim checks; SQLite supports parameter binding and transactions. Application code still defines ownership, role semantics and delegation. Deployment choices determine network exposure and process isolation.

6 Verification and continuous integration

The final local validation contains 40 passing tests. Functional tests cover create, list, read, export, result retrieval and metadata auditing. Security tests compare owners, roles and credentials through real HTTP calls. Four artifact tests verify accepted bytes and rejection of changed, missing or unexpected files. Fixtures create a temporary database and fresh services, so test success does not depend on previous demonstration state. Saved test output and JUnit XML identify the executed cases and outcomes.

The tests call both the public API and the internal service endpoints. They supply legitimate test grants with the wrong subject, action, audience or dataset, attempts a Resource credential signed using Processor's key, submits altered CSV output and calls internal routes without authentication. The tests use temporary keys to construct these cases; no live credentials are published.

scripts/ci.py performs artifact verification, installed-distribution verification, compilation, a helper wheel build, Ruff lint and formatting, Bandit, pip-audit, the test suite and a smoke demonstration. It records every stage's exit status in reports/gates.json. Any failure blocks the candidate. The final snapshot records zero for every stage. Ruff and Bandit report no remaining findings in their configured scope. Bandit examines application and helper code; test assertions and the process-launching development harness are outside that security scan.

Setup downloads approved dependency versions and checks their SHA-256 hashes. The small local helper wheel is verified separately. The helper manifest rejects unexpected files, missing files and changed bytes. A separate check compares installed names and versions with the approved list and verifies that the helper source matches its approved wheel. Any known public dependency vulnerability blocks the gate. The local helper has no public advisory entry, so its documented skip is covered by source inspection, static analysis and artifact checks. Unknown skipped public packages and advisory-service failures block.

The Jenkinsfile starts with a fresh checkout on a Windows agent with Python 3.12, installs approved artifacts and executes those same local gates. It publishes JUnit results and archives analysis output even after failure. A GitHub Multibranch Pipeline with branch and pull-request discovery plus webhook configuration is required to schedule builds on both kinds of event [8]. This account-level configuration has not been performed here. The separate Windows submission guide explains this setup; actual hosted build URLs must be added afterward.

The results have limits. Linux execution and fresh setup were verified. PowerShell commands and approved Windows package hashes are supplied, but a Windows run was unavailable. The tests do not prove freedom from all vulnerabilities, resistance to sustained load, or secure operation after complete host compromise. Dependency advisories are a dated observation and must be checked again when the project changes.

Saved evidence	Outcome
evidence/final/tests.xml and tests.txt	40 tests passed
evidence/final/gates.json	All 10 stages exit 0
evidence/final/ruff.json and bandit.json	No configured findings
evidence/final/dependency-audit.json	No known public dependency advisories
evidence/vulnerable-tests.txt	Four expected security assertion failures

7 Release considerations and submission

The final implementation supports the required small workflow while correcting the demonstrated security failures. Its central result is that delegation preserves the user and requested dataset through every service boundary, and Resource independently enforces ownership before data leaves storage or a result is saved. A successful authenticated call is evidence of identity, not a universal access permission. This distinction explains both the original direct-read flaw and the separate export-composition flaw.

For production, the loopback-only deployment would need TLS and authenticated transport between services, separate operating-system identities or containers, restricted database and account-file permissions, and controlled outbound network access. Signing keys need protected storage, rotation and revocation procedures. HMAC verification keys also permit signing within their trust domain; asymmetric credentials would reduce that shared authority. Public demonstration passwords must be replaced, and login requires throttling and appropriate account protections.

Availability work must include incoming byte limits before JSON parsing, database growth quotas, concurrency and load testing, backup restoration and operational monitoring. Current row limits bound accepted datasets but do not prevent a client from sending an enormous HTTP body. Current logs provide metadata evidence for local investigation, not a complete durable audit system. Admin-visible titles must remain nonsensitive metadata. Updating dependencies also requires reviewing new artifacts and refreshing advisory evidence.

On Windows, the root run.py launcher keeps the procedure short. The setup command creates the virtual environment, verifies and installs dependencies, and seeds data. The run command starts all three services. The test command saves test results; check performs all release gates and saves analysis output. README.md provides the exact PowerShell commands, while docs/github_and_submission.md covers publishing and Jenkins. Activation is unnecessary.

The supplied repository includes full Git history, the initial plan, architecture, access matrix, helper, Jenkinsfile, regression evidence and operating instructions. Submission requires publishing that history, configuring Jenkins and retaining actual push and pull-request build links. Student number, repository address and build links remain to be completed. The declaration is a separate attachment. No remote publication or student review is claimed.

The practical oral-exam preparation is to trace one authorized export, repeat the cross-user denial, inspect the vulnerable tag and explain why each check belongs at its particular boundary.

Windows PowerShell command	Purpose
py -3.12 run.py setup	Create environment, install and seed
py -3.12 run.py run	Start all three services
py -3.12 run.py demo	Demonstrate a running application
py -3.12 run.py test	Run tests and save JUnit results
py -3.12 run.py check	Run all checks and save analysis

References and evidence navigation

[1] Software Security Assignment I. Supplied course assignment, including required architecture, SSDLC, pipeline and report requirements.

[2] Student Reading Material — Software Security. Supplied CWE catalog, sections CWE-20, CWE-89 and CWE-862.

[3] Abhishek Vaish. Lesson 2 Security Design Principle Why Security Fails. Supplied lecture, especially least privilege, fail safe, minimization and KISS.

[4] Abhishek Vaish. Software Vulnerabilities and Analyzing Software Vulnerabilities. Supplied lectures on assumptions, trigger conditions and vulnerability analysis.

[5] Unit 1 DevSecOps and From Threat Modeling to a DevSecOps Pipeline. Supplied lecture and exercise, lifecycle activities and gate traceability.

[6] Yuning Jiang et al. A Survey on Vulnerability Prioritization Taxonomy Metrics and Research Challenges. arXiv:2502.11070v1, 2025. Supplied reading, contextual prioritization.

[7] PyJWT. Usage Examples. https://pyjwt.readthedocs.io/en/latest/usage.html Accessed 22 October 2026. Claim and audience validation.

[8] Jenkins. Branches and Pull Requests. https://www.jenkins.io/doc/book/pipeline/multibranch/ Accessed 22 October 2026. Multibranch discovery.

[9] Aarhus University. Generative artificial intelligence. https://studerende.au.dk/en/gai Accessed 22 October 2026. Separate AI-use declaration accompanies this draft.

Code and saved evidence

Repository-relative references remain usable before a GitHub URL exists. See workspace_app/api.py for login and delegation; common.py for token validation; resource.py for owned, job_read and job_save; processor.py for the installed helper call. tests/test_http.py covers the four named probes; tests/test_grants.py exercises internal boundaries; tests/test_artifacts.py exercises artifact rejection. docs/security_design.md contains the full STRIDE and access matrix.

History references: faf9582 initial plan; dcabff9 vulnerable-lab; 716e188 fixed-boundaries. See evidence/commit_map.txt for subsequent commits and evidence/final/ for the final local results. The live hosted build fields below must be completed after account setup; no hosted execution is claimed.

Submission field	Complete before submission
Student name and number	Moneer | [Student number]
Repository URL	[Complete]
Jenkins push build URL	[Complete after actual execution]
Jenkins pull request build URL	[Complete after actual execution]

Main report narrative: 2495 words, excluding headings, tables, diagrams, references and this navigation page.